

School of Science

DEPARTMENT OF MATHEMATICS
Master's Degree in Data Science

Implementation of batch methods to solve a multi-class classification problem

Professor
Francesco Rinaldi

Students

Daniele Barolo (2078339)
Riccardo Cazzin (2130959)
Anna Putina (2081379)
Sara Zampelli (2105691)

Academic Year 2023–24

Contents

1	Introduction	1
2	Datasets	1
2.1	Synthetic dataset	2
2.2	Wine dataset	2
2.3	Estimation of Obesity dataset . .	2
2.4	Digits dataset	2
3	Methods	2
3.1	Gradient descent variants	3
3.2	Line search criteria	3
4	Practical results	4
4.1	Performance Metrics	4
4.2	Fixed Learning Rate vs Armijo Rule	4
4.3	Parameter Selection	7
4.4	Gradient Descent vs Modifications	7
5	Conclusions	7
	References	8

1 Introduction

Multiclass logistic regression stands as one of the algorithms in machine learning used to handle classification tasks where the number of classes is greater than two. The extension of the binary logistic regression model allows robust assignment of observations to any one among several predefined classes, making it possible through a negative log-likelihood function optimization for class probability prediction and hence decision making on unseen data. This paper hunts through the theoretical framework surrounding multiclass logistic regression, with a couple practical applications.

The optimization problem is mathematically formulated as the minimization of a negative log-likelihood function where parameters are optimized to maximize the likelihood of observing the training data. Mathematically, this optimization problem can be framed as

$$\min_{x \in \mathbb{R}^{d \times k}} \sum_{i=1}^m \left[-x_{b_i}^T a_i + \log \left(\sum_{c=1}^k \exp(x_c^T a_i) \right) \right] \quad (1.1)$$

Here, X represents the parameter matrix of dimension $d \times k$, a_i denotes the feature vector of the i -th training example in $\mathbb{R}^d$, and b_i denotes its corresponding class label within the range $\{1, 2, \dots, k\}$.

The likelihood for a single training example i is given by:

$$P(b_i | a_i, x) = \frac{\exp(x_{b_i}^T a_i)}{\sum_{c=1}^k \exp(x_c^T a_i)} \quad (1.2)$$

The negative log-likelihood function is minimized across all training examples:

$$f(x) = \sum_{i=1}^m \left[-x_{b_i}^T a_i + \log \left(\sum_{c=1}^k \exp(x_c^T a_i) \right) \right] \quad (1.3)$$

This research intends to explore and compare three gradient-based optimization techniques for solving the minimization problem in (1.1). Additionally, it seeks to evaluate these techniques' performance in both computational efficiency and prediction accuracy, aiming to provide valuable insights for the optimization of multiclass logistic regression. A wide range of experiments will be conducted as part of this study, from synthetic data generation to real-world dataset analysis.

Through this work, this research aims to shed light on multiclass logistic regression optimization, offering valuable insights for both theoretical developments and practical.

2 Datasets

For our study, we used four distinct datasets:

1. a synthetic (toy) dataset;
2. the Wine dataset;
3. the Estimation of Obesity dataset;
4. the Digits dataset.

Each dataset provides unique characteristics that are crucial for evaluating the performance of the optimization techniques. For the purpose of analysis, datasets 2, 3, and 4 were split into training and testing sets using an 80-20 split ratio.

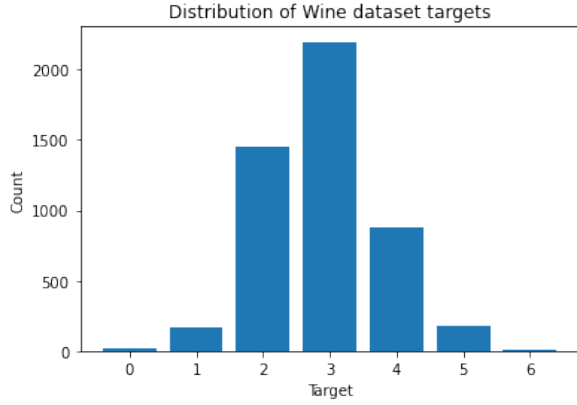


Figure 2.1: Distribution of the Wine dataset’s targets.

2.1 Synthetic dataset

The synthetic dataset is generated through the following process:

- randomly generate a 1000×1000 matrix with entries from a $\mathcal{N}(0, 1)$ distribution;
- generate $b_i \in \{1, \dots, k\}$ (where $k = 50$) by computing $AX + E$ with $X \in \mathbb{R}^{d \times k}$ and $E \in \mathbb{R}^{m \times c}$ sampled from a normal distribution. The maximum index in the row is considered as the class label.

2.2 Wine dataset

The Wine dataset [3] comprises data from *vinho verde*, a product from the Minho region of Portugal. Collected from May 2004 to February 2007 by the Comissão de Viticultura da Região dos Vinhos Verdes (CVRVV) [2], the dataset includes various physicochemical tests (see Table 1). The sensory assessors gave scores between 3 and 9, which were converted to labels between 0 and 6, with 0 being very bad and 6 being very good. The distribution of the quality of the samples can be seen in Figure 2.1.

2.3 Estimation of Obesity dataset

The Estimation of Obesity dataset [6] consists of 2111 records and 17 attributes collected from undergraduate students aged 18-25 from Colombia, Mexico, and Peru [5]. The data was gathered through surveys capturing factors such as age, weight, sex, physical activity, and fast food intake to assess obesity levels (see Table 2).

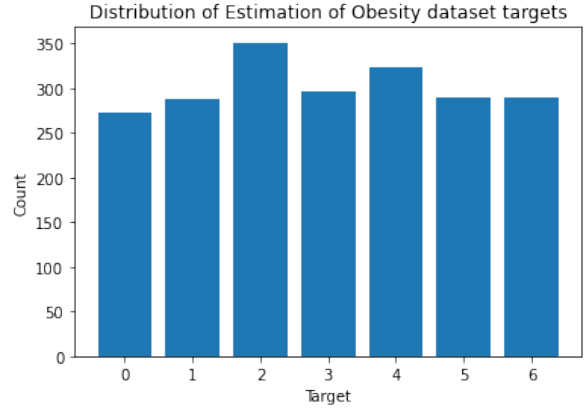


Figure 2.2: Distribution of the Estimation of Obesity Dataset’s targets. Initially, the dataset showed an imbalance in the categories of obesity levels. To rectify this, synthetic data was generated by the authors using the SMOTE technique in Weka, balancing up to 77% of the dataset [5].

The target variable, obesity level, is based on the Body Mass Index (BMI) formula calculated as $\text{Weight}/\text{Height}^2$. It was categorized into *underweight* (<18.5), *normal* ($18.5\text{--}24.9$), *overweight* ($25.0\text{--}29.9$), *obesity I* ($30.0\text{--}34.9$), *obesity II* ($35.0\text{--}39.9$), and *obesity III* (>40) based on WHO and Mexican standards.

2.4 Digits dataset

The Digits dataset comprises 1797 samples, each represented by a 8×8 pixels image of a handwritten digit. Each pixel in these images can take an integer value between 0 (black) and 16 (white), indicating the pixel’s intensity (see Table 3). The label of each sample corresponds to the digit represented in the image. The distribution of such labels can be seen in Figure 2.3.

The Digits dataset was downloaded using the `sklearn.datasets.load_digits` package and is a copy of the test set of the UCI ML hand-written digits dataset [1].

3 Methods

In order to solve the minimization problem in (1.1), the following iterative methods will be used:

- the standard *gradient descent* method;

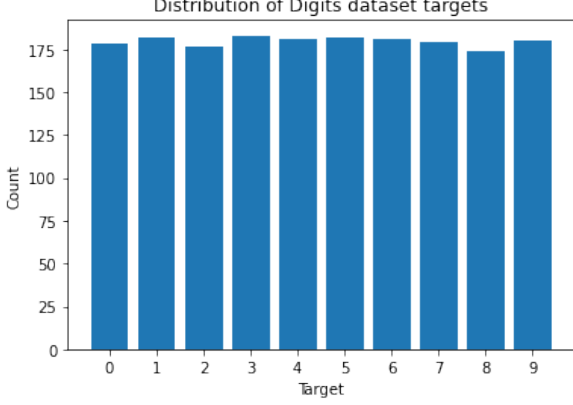


Figure 2.3: Distribution of the Digits Dataset’s targets.

- the BCGD, i.e. *block coordinate gradient descent*, method, with two different block choice rationales:
 - random block choice;
 - Gauss–Southwell rule.

These methods will present different trade-offs in both CPU time and number of iterations.

Since the above methods are usually presented for optimization problems over standard vector spaces, in this section all matrices of the form $X \in \mathbb{R}^{d \times k}$ (as the matrix X in the problem statement) will be considered as vectors $x \in \mathbb{R}^D$ (with $D = dk$) obtained by juxtaposing the rows of X in order.

The results in §4 will also compare two different line search criteria:

- fixed step size;
- Armijo’s rule.

Further details about all of these technical devices can be found in Nesterov [4, §1.2.3].

3.1 Gradient descent variants

Given a starting point $x_1 \in \mathbb{R}^D$, the standard gradient descent method involves using the update rule

$$x_{\kappa+1} = x_{\kappa} - \alpha_{\kappa} \nabla f(x_{\kappa}) \quad (3.1)$$

until a certain condition on x_{κ} is satisfied. Theoretically, the condition should be $\nabla f(x_{\kappa}) = \mathbf{0}$, but machine precision makes this condition quite unrealistic to achieve. In §4, the proposed conditions will be

- $\kappa \leq \bar{\kappa}$ for a convenient $\bar{\kappa} \in \mathbb{N}$;
- $\|\nabla f(x_{\kappa})\| \leq \varepsilon$ with $\varepsilon > 0$ small enough.

Both these conditions will be tuned by hand, depending on the performance of the method on the available data.

The two BCGD variants rely on the choice of blocks of entries of x . In the cases proposed in §4, these blocks will correspond to a single row of X , which means picking the consecutive entries $i, \dots, i + k - 1$ of x , provided that $i \equiv 1$ modulo k and $i \leq D - k + 1$. If U_i represents the standard embedding of the above mentioned block of variables into the “full” space $\mathbb{R}^D$ and U_i^T is the standard projection of $\mathbb{R}^D$ onto the space associated to the block of variables, $\nabla_i f(x) = U_i^T \nabla f(x)$ denotes the vector of partial derivatives restricted to the above mentioned variables; the update rule associated with the BCGD method, then, can be written as

$$x_{\kappa+1} = x_{\kappa} - \alpha_{\kappa} U_i \nabla_i f(x_{\kappa}) \quad (3.2)$$

The stopping conditions that will be used are the same as for (3.1).

Two rules were chosen to select the index i , and therefore the corresponding block, at each iteration. The first rule involves choosing a suitable i at random, sampling from a discrete uniform distribution. The second rule is the Gauss–Southwell rule, which chooses i such that

$$i = \operatorname{argmax} \left\{ \|\nabla_j f(x)\| : \begin{cases} j \equiv 1 \pmod{k} \\ j \leq D - k + 1 \end{cases} \right\} \quad (3.3)$$

Note that this selection rule generally requires the knowledge of the full gradient to pick the most suitable index.

3.2 Line search criteria

The update rules (3.1)–(3.2) require the knowledge or computation of a step size α_{κ} for each update.

The easiest approach is to fix a step size $\alpha_{\kappa} \equiv \alpha$ for all iterations of the method, i.e. for all $\kappa \in \mathbb{N}$. Exact theoretical methods to choose the best step size based on the properties of the function f exist; however, they require the knowledge of some parameters (e.g. the Lipschitz constant of ∇f) which are costly to compute in practice, as their computation

involves the use of higher-order derivatives. For this reason, experiments in §4 will make use of hand-tuned fixed step sizes, which should avoid over-shooting the target stationary point anyway.

The other proposed approach is Armijo’s rule. This method yields a (potentially) different step size at each iteration, based on its implied performance in that specific point. In practice, Armijo’s rule requires two parameters $\delta \in (0, 1)$ and $\gamma \in (0, 1/2)$ and a starting step size Δ ; given $\alpha = \delta^m \Delta$ for increasingly large $m \in \mathbb{N}$, if

$$f(x_\kappa + \alpha \nabla f(x_\kappa)) \leq f(x_\kappa) + \gamma \alpha \|\nabla f(x_\kappa)\|^2 \quad (3.4)$$

then $\alpha_\kappa = \alpha$ —this means that the smallest m that verifies (3.4) is used. Since this line search criterion involves many computations of f and ∇f , each iteration can be computationally costly, especially when dealing with large-scale problems. In §4, unless specified otherwise, the constants $\gamma = 1/4$, $\delta = 1/2$ and $\Delta = 1$ will be chosen.

4 Practical results

In this section, we analyze our experiments comparing different gradient descent approaches: Ordinary Gradient Descent, Block Coordinate Gradient Descent with Randomized selection, and BCGD with Gauss–Southwell selection. Each method was tested using both fixed learning rates and adaptive learning rates guided by the Armijo rule.

The visual results for all datasets in §2 are presented below. To make the plots more representative, we stick to using the maximum iterations boundary rather than the stopping conditions.

Overall, we managed to reach convergence in 3 out of 4 datasets: Toy, Obesity, and Digits. For the Digits and Toy datasets, we obtained near 100 % accuracy, which was not the case for the Obesity dataset. This can be explained by the artificial nature of the first two datasets, while the Obesity dataset consists of real data. For the Wine dataset, all the methods did not converge within a reasonable number of iterations, because it was impossible to grasp the complexity of the data using the loss function we had for this task. More complex ones have

the potential to solve this problem, but it is out of the scope of this project.

Next, we discuss the general results and patterns of behavior observed in the methods under investigation. We do not delve into specifics for each dataset since the results are consistent across them, although we reference the data we used.

4.1 Performance Metrics

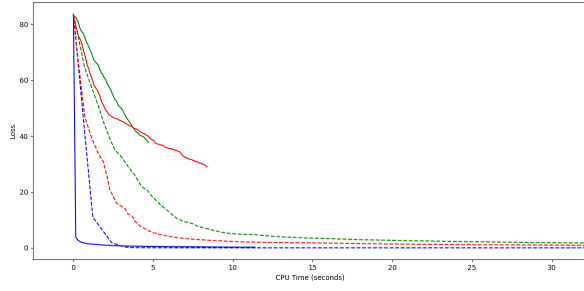
- **Convergence Time:** Evaluated as the total computational time until convergence.
- **Number of Iterations:** The total iterations required to reach the convergence criterion.
- **Final Loss Value:** The value of the loss function at convergence.
- **Stability:** Assessed by the algorithm’s robustness in different initial conditions and datasets.

4.2 Fixed Learning Rate vs Armijo Rule

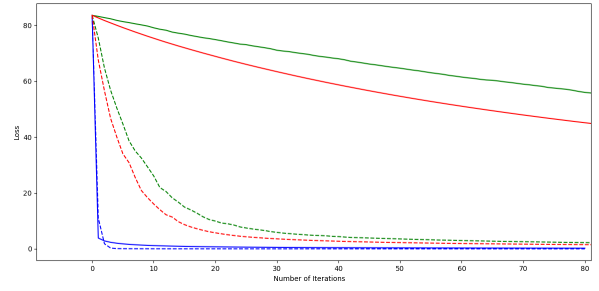
All variants of gradient descent converge faster in terms of time with a fixed learning rate compared to the Armijo rule, although the latter requires fewer iterations. This behavior was observed across all datasets under investigation. It could be attributed to the dynamic adjustment of the learning rate when using the Armijo rule, which consumes more time. It is worth noting that accuracy could be slightly better with the Armijo rule.

The parameter γ in the Armijo rule should be chosen carefully. If it is too small, the loss function could become trapped in a local minimum, particularly when using Gauss–Southwell’s systematic selection. The constant learning rate may not always align with the local curvature of the loss surface, leading to slower convergence especially in areas with shallow gradients.

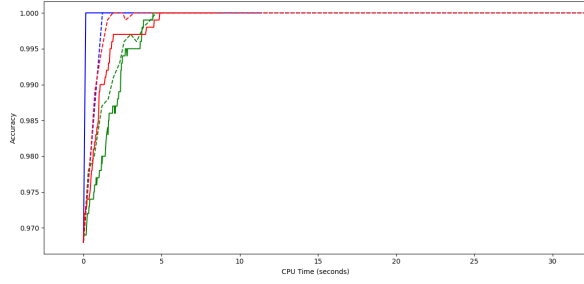
Our cases were manageable with a fixed learning rate, although it doesn’t always perform well. Sometimes, it fails to adjust to the changing landscape of the gradient, resulting in more iterations needed to reach the optimal point, especially in areas with shallow gradients.



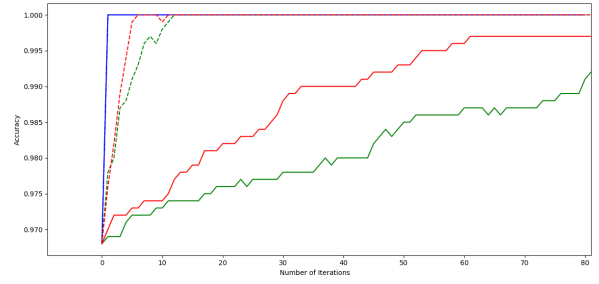
(a) Loss as a function of CPU time.



(b) Loss as a function of the number of iterations.

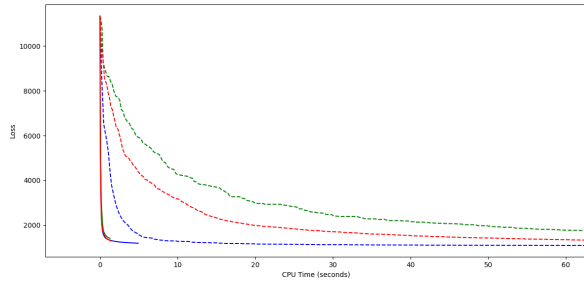


(c) Accuracy as a function of CPU time.

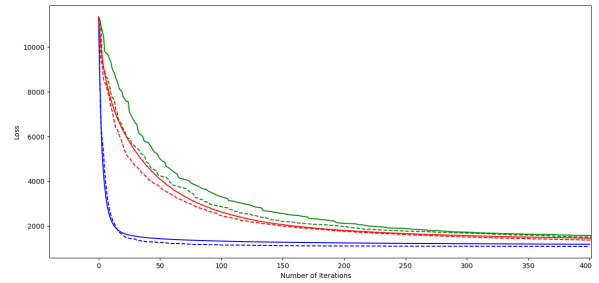


(d) Accuracy as a function of the number of iterations.

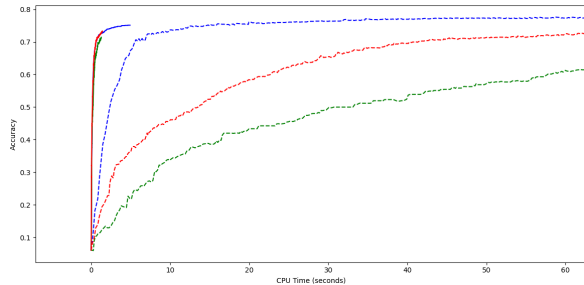
Figure 4.1: Performance of methods for the toy dataset. Blue lines show the results for the standard gradient descent; green lines are for the randomized BCGD method; red lines are for the Gauss–Southwell BCGD method. A continuous line is for a fixed learning rate, whereas the dashed lines are for the results with the Armijo rule.



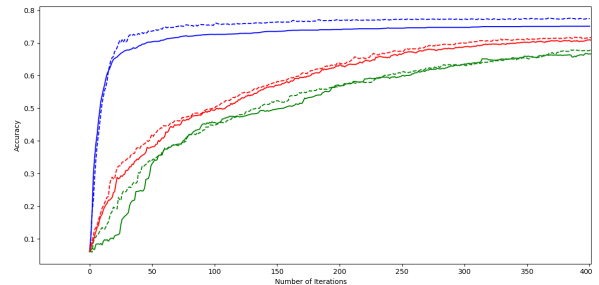
(a) Loss as a function of CPU time.



(b) Loss as a function of the number of iterations.

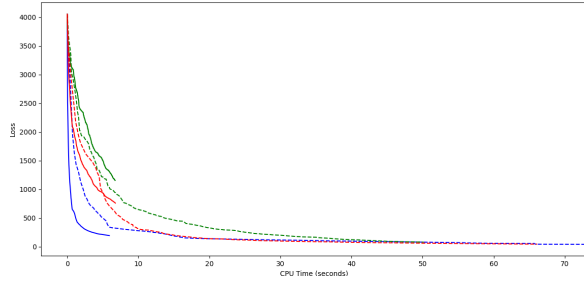


(c) Accuracy as a function of CPU time.

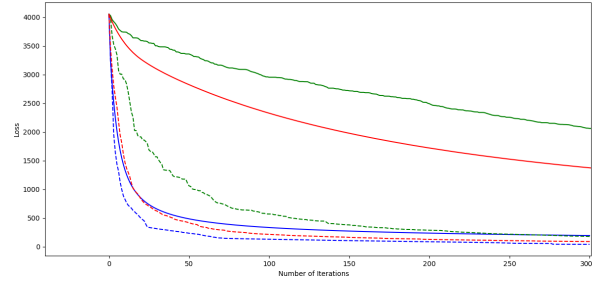


(d) Accuracy as a function of the number of iterations.

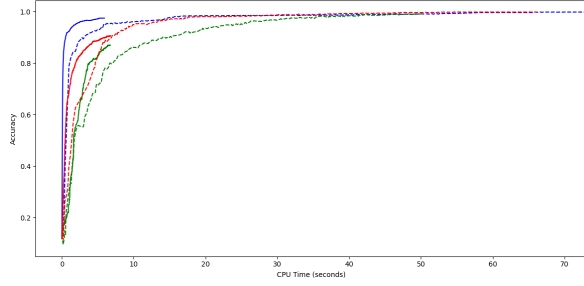
Figure 4.2: Performance of methods for the Obesity dataset. Blue lines show the results for the standard gradient descent; green lines are for the randomized BCGD method; red lines are for the Gauss–Southwell BCGD method. A continuous line is for a fixed learning rate, whereas the dashed lines are for the results with the Armijo rule.



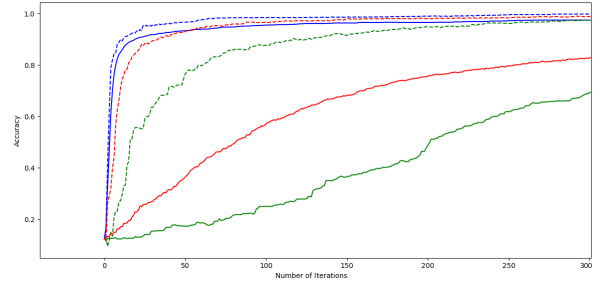
(a) Loss as a function of CPU time.



(b) Loss as a function of the number of iterations.

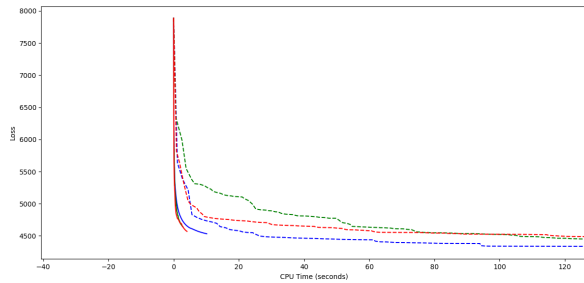


(c) Accuracy as a function of CPU time.

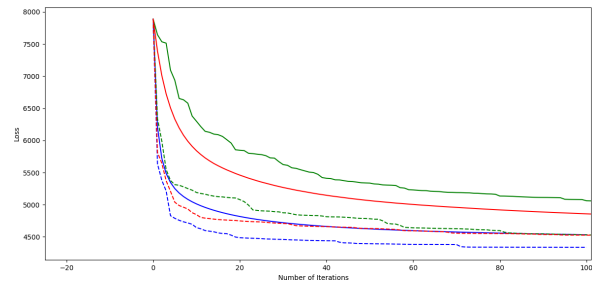


(d) Accuracy as a function of the number of iterations.

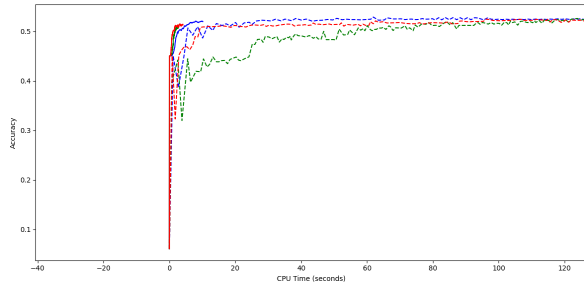
Figure 4.3: Performance of methods for the Digits dataset. Blue lines show the results for the standard gradient descent; green lines are for the randomized BCGD method; red lines are for the Gauss–Southwell BCGD method. A continuous line is for a fixed learning rate, whereas the dashed lines are for the results with the Armijo rule.



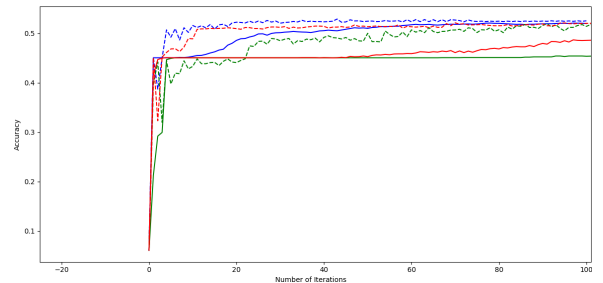
(a) Loss as a function of CPU time.



(b) Loss as a function of the number of iterations.



(c) Accuracy as a function of CPU time.



(d) Accuracy as a function of the number of iterations.

Figure 4.4: Performance of methods for the Wine dataset. Blue lines show the results for the standard gradient descent; green lines are for the randomized BCGD method; red lines are for the Gauss–Southwell BCGD method. A continuous line is for a fixed learning rate, whereas the dashed lines are for the results with the Armijo rule.

4.3 Parameter Selection

While performing the task, we faced several instances of parameter selection: learning rate, number of iterations, gradient tolerance, patience for the loss function, and parameters for the Armijo rule. There are two general practices for choosing these parameters: relying on theoretical foundations and adjusting through exploration of the algorithm’s behavior on a specific dataset.

A good example is the learning rate for our project. According to theory, it should be set as the Lipschitz constant, which requires Hessian calculation. However, this is not feasible for some tasks and was particularly difficult for ours. Therefore, we chose the learning rate by analyzing the algorithm’s performance: observing the decreasing pattern of the loss, checking for fluctuations, and assessing the pace of the decrease.

The same approach was applied to the gradient tolerance. The absolute value of the gradient highly depends on the nature of the data, so we set it after considering the behavior of each algorithm.

4.4 Gradient Descent vs Modifications

For our relatively simple situation, ordinary gradient descent proved to be faster compared to its modifications. This can be attributed to the greater complexity of the other methods. Both randomized and Gauss–Southwell BCGD work with only one block at a time, which turned out to be less effective and accurate for our data.

Although our results favored ordinary gradient descent, it is important to note that this is not always the case. In more complex scenarios, where the optimization landscape is more challenging, BCGD methods can perform better. For example, in high-dimensional problems with sparse gradients, BCGD can efficiently handle sub-problems and converge faster than ordinary gradient descent. Additionally, BCGD can be more effective in problems where the variables are naturally partitioned into blocks, each of which can be optimized separately. Ordinary gradient descent may struggle in these situations due to its inability to exploit such structure, leading to slower convergence and

higher computational costs.

Another interesting discovery is that Gauss–Southwell BCGD tends to work faster in our scenario. This indicates that the cost of gradient calculation is significant compared to the cost of selecting the block with the largest gradient. Additionally, randomized BCGD is slower because the block selection is random and, therefore, not always optimal.

5 Conclusions

In this study, we explored the performance of different gradient descent approaches using both fixed learning rates and adaptive learning rates guided by the Armijo rule. Our experiments across four datasets provided several insights.

Firstly, we observed that the choice of optimization techniques is heavily dependent on the task at hand and the resources available, such as computational time and CPU capacity. Ordinary Gradient Descent often outperformed the more complex BCGD methods in our relatively simple scenarios, primarily due to its straightforward approach. However, in more complex, high-dimensional problems with sparse gradients, BCGD methods, and the Gauss–Southwell variant in particular, can be more efficient.

Our results showed that methods with fixed learning rates generally converged faster in terms of computational time compared to those using the Armijo rule, despite requiring more iterations. This is due to the additional computational overhead involved in dynamically adjusting the learning rate using the Armijo rule, though it may lead to slightly higher accuracy. The parameter γ in the Armijo rule must be carefully chosen to avoid local minima, especially in systematic selection methods like Gauss–Southwell.

The data also played a significant role in the optimization performance. Uniformly distributed labels generally led to more balanced model performance, whereas imbalanced distributions, as seen in the Wine dataset, could bias the model towards more frequent classes, potentially degrading performance on less frequent classes.

Finally, we opted for hand-tuned step sizes as learning rates instead of theoretical Lips-

chitz constants, which are impractical to compute in real-world scenarios, especially in high-dimensional settings. This pragmatic approach allowed us to tailor the learning rates based on observed performance and ensure a more adaptive and efficient optimization process.

In summary, our findings emphasize the importance of selecting the appropriate optimization technique based on the specific task requirements and resource constraints, and highlight the impact of data distribution and practical parameter selection in the effectiveness of gradient descent methods.

References

- [1] E. Alpaydin and C. Kaynak. *Optical Recognition of Handwritten Digits*. June 30, 1998. URL: <https://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits>.
- [2] P. Cortez et al. “Using data mining for wine quality assessment”. In: *Discovery Science* (Oct. 2009), pp. 66–79. DOI: 10.1007/978-3-642-04747-3_8.
- [3] P. Cortez et al. *Wine Quality dataset*. Oct. 6, 2009. URL: <https://archive.ics.uci.edu/dataset/186/wine+quality>.
- [4] Yurii Nesterov. *Introductory Lectures on Convex Optimization. A Basic Course*. Springer, 2013. ISBN: 978-1-4419-8853-9.
- [5] Fabio Mendoza Palechor and Alexis De la Hoz Manotas. “Dataset for estimation of obesity levels based on eating habits and physical condition in individuals from Colombia, Peru and Mexico”. In: *Data in Brief* 25 (2019). DOI: 10.1016/j.dib.2019.104344.
- [6] Fabio Mendoza Palechor and Alexis De la Hoz Manotas. *Estimation of Obesity Levels Based On Eating Habits and Physical Condition*. Aug. 26, 2019. URL: <https://archive.ics.uci.edu/dataset/544/estimation+of+obesity+levels+based+on+eating+habits+and+physical+condition>.

Table 1: Wine Dataset variables description.

Attribute	Minimum	Maximum	Mean
fixed acidity (g(tartaric acid)/dm ³)	3.80	14.20	6.85
volatile acidity (g(acetic acid)/dm ³)	0.08	1.10	0.28
citric acid (g/dm ³)	0.00	1.66	0.33
residual sugar (g/dm ³)	0.60	65.80	6.39
chlorides (g(sodium chloride)/dm ³)	0.01	0.35	0.05
free sulfur dioxide (mg/dm ³)	2.00	289.00	35.31
total sulfur dioxide (mg/dm ³)	9.00	440.00	138.36
density (g/cm ³)	0.99	1.04	0.99
pH	2.72	3.82	3.19
sulphates (g(potassium sulphate)/dm ³)	0.22	1.08	0.49
alcohol (% vol.)	8.00	14.20	10.51

Table 2: Estimation of Obesity Dataset variables description.

Attribute	Min	Max	Mean
Age	14.00	61.00	24.31
Gender (0: Female, 1: Male)	0.00	1.00	
Height (meters)	1.45	1.98	1.70
Weight (kilograms)	39.00	173.00	86.59
High caloric food frequency (0: No, 3: Always)	0.00	3.00	
Frequent consumption of vegetables (0: No, 1: Yes)	0.00	1.00	
Consumption of food between meals (1: Sometimes, 3: Frequently)	1.00	3.00	
Number of main meals (1: Between 1 and 2, 4: More than three)	1.00	4.00	
Consumption of food between meals (0: No, 1: Yes)	0.00	1.00	
Smoking habit (0: No, 1: Yes)	0.00	1.00	
Daily water consumption (1: Less than a liter, 3: More than 2 L)	1.00	3.00	
Monitoring calorie intake (0: No, 1: Yes)	0.00	1.00	
Physical activity frequency (0: I do not have, 5: Always)	0.00	5.00	
Time spent on technological devices (0: 0-2 hours, 3: More than 5 hours)	0.00	3.00	
Frequency of alcohol consumption (0: I do not drink, 3: Always)	0.00	3.00	
Transportation method (0: Automobile, 4: Walking)	0.00	4.00	

Table 3: Digits Dataset summary.

Property	Value
Classes	10
Samples per class	~180
Samples total	1797
Dimensionality	64
Features	integers 0–16

